ROSE5710: Fundamental Robotics and Systems **2026–27 First Term**

Assignment 1: Kinematics

Instructor: LAU Darwin Tat Ming Due: November 13, 2026

Assignment Overview & Objectives:
In this assignment, you will analyze the Open Duck Mini Bipedal Robot. The tasks are divided into three main parts:

1. **Denavit-Hartenberg (D-H) Derivation:** You will analyze the zero-pose geometry using the link lengths (d_i) and joint angles (θ_i) annotated on the diagrams. Your task is to assign coordinate frames according to Modified D-H convention and extract the D-H parameters for the left leg, right leg, and head/neck chains.
2. **Numerical Inverse Kinematics:** Using your FK model, you will define a position-and-orientation error objective, estimate its gradient using finite differences, and update the joint angles using fixed-step gradient descent within the joint limits.
3. **Software Implementation & Verification:** You will map your derived D-H parameters and the supplied dimensions to the Python template. You will verify Forward Kinematics (FK) at random joint configurations and use your IK solver to track predefined task-space trajectories. Compare both position and orientation with the site poses reported by MuJoCo.

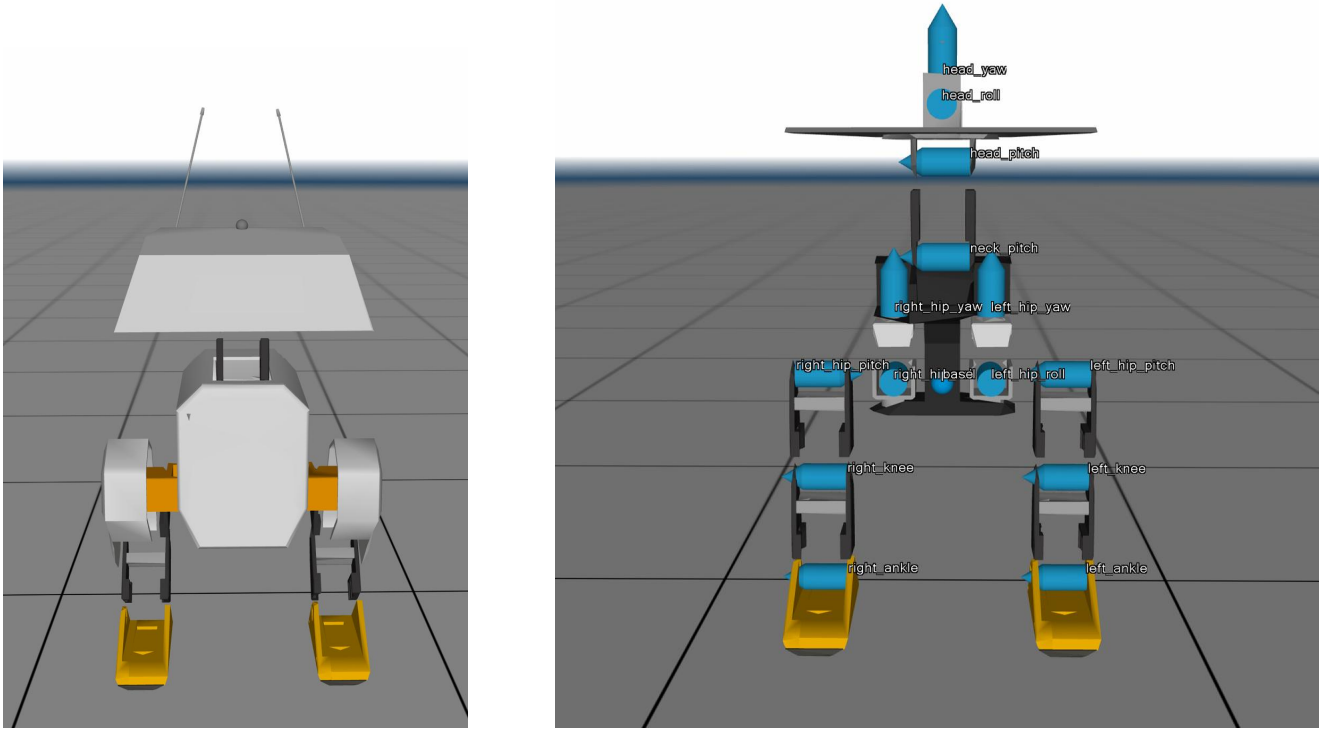


Figure 1: External view of the Robo Duck platform and the internal configuration of its kinematic joints and links.

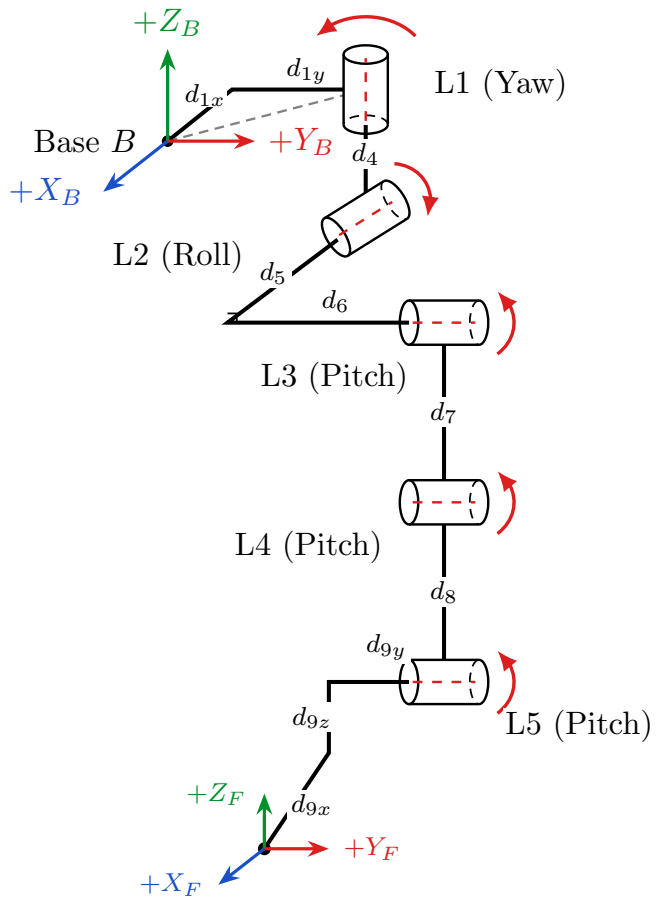
1 Forward kinematics: complete `forward_kinematics.py`

Location	Your work
`forward_kinematics_transforms()`	TODO 1: combine each row's four existing matrix helpers with the running transform. Append the result after every row.
`right_leg_parameters()`	TODOs 2–3: write the right-leg table (six rows) and five joint signs.
`left_leg_parameters()`	TODOs 4–5: write the left-leg table (six rows) and five joint signs.
`head_parameters()`	TODOs 6–7: write the head table (five rows) and four joint signs.

For TODO 1, use the angle already computed in the loop. For the parameter functions, use the figures below, convert millimetres to metres, and include one final fixed row for each site.

1.1 Left leg (5 DOF)

Use the figure and dimensions to complete **TODOs 4–5** in `left_leg_parameters()`. Return a 6×4 `dh_table`, five `signs`, and the supplied `T_base_0`.



Leg dimensions (mm)

Label	Left leg	Right leg
d_{1x}	19.000	19.000
d_{1y}	35.000	35.000
d_4	0.0741	0.0741
d_5	-16.060	-16.060
d_6	74.150	74.150
d_7	78.650	78.650
d_8	78.650	78.650
d_{9y}	19.750	17.350
d_{9z}	36.225	36.225
d_{9x}	0.500	0.500

Figure 2: Left-leg chain and dimensions for both legs.

Table 1: Left-leg D-H table. Row 6 is the fixed site row.

i	α_{i-1}	a_{i-1}	d_i	θ_i
1				
2				
3				
4				
5				
6 (fixed)				

1.2 Right leg (5 DOF)

Use the mirrored figure and the dimensions beside Figure 2 to complete **TODOs 2–3** in `right_leg_parameters()`. Return a 6×4 `dh_table`, five `signs`, and the supplied `T_base_0`.

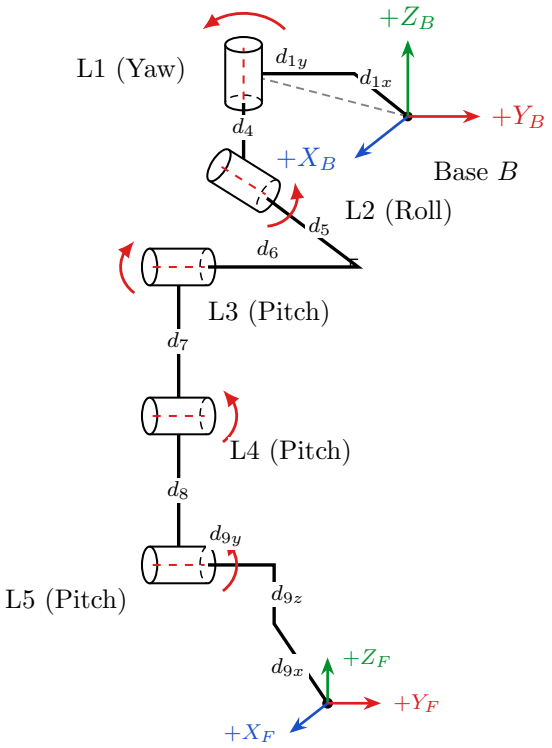


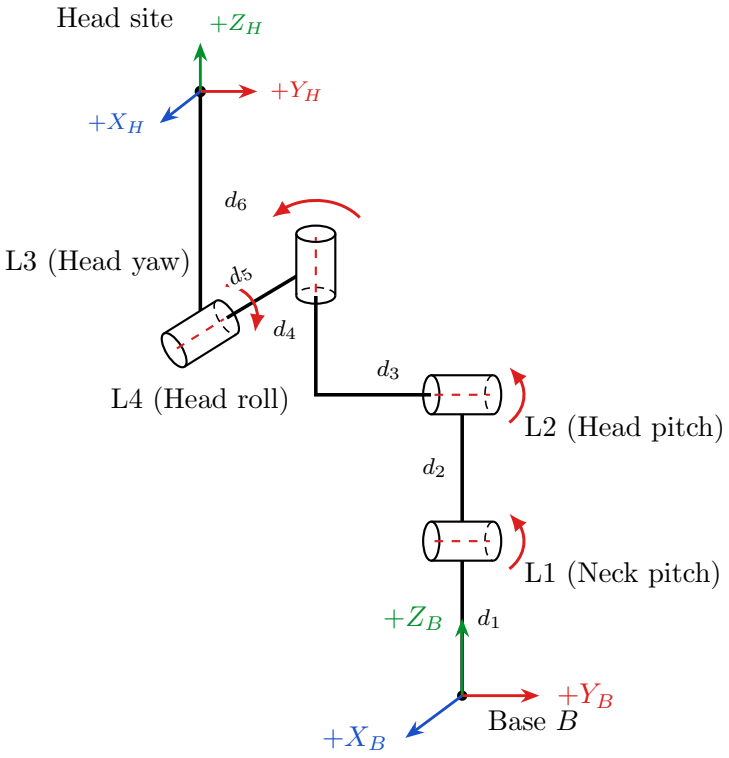
Figure 3: Right leg chain.

Table 2: Right-leg D-H table. Row 6 is the fixed site row.

i	α_{i-1}	a_{i-1}	d_i	θ_i
1				
2				
3				
4				
5				
6 (fixed)				

1.3 Head and neck (4 DOF)

Use the figure and dimensions to complete **TODOs 6–7** in `head_parameters()`. Return a 5×4 `dh_table`, four `signs`, and the supplied `T_base_0`.



Head dimensions (mm)

Label	Length (mm)
d_1	92.021
d_2	66.000
d_3	19.050
d_4	37.850
d_5	5.000
d_6	42.450

Figure 4: Head chain and dimensions.

Table 3: Head D-H table. Row 5 is the fixed site row.

i	α_{i-1}	a_{i-1}	d_i	θ_i
1				
2				
3				
4				
5 (fixed)				

2 Inverse kinematics: complete `inverse_kinematics.py`

Complete these three TODOs after the FK code. Replace each `NotImplementedError` with your implementation. All target and predicted poses use the base frame; joint angles are in radians and translations are in metres.

1. **TODO 1, `objective()`:** Compute the target-minus-predicted position error e_p and the rotation vector e_R of $R_{\text{target}}R_{\text{FK}}^{\mathsf{T}}$. Return the **weighted squared L2 error**:

$$E(q) = \|s_p e_p\|_2^2 + \|e_R\|_2^2, \qquad s_p = \texttt{position_scale}.$$

   The result is a single nonnegative cost for gradient descent to minimize.

2. **TODO 2, `inverse_kinematics()`:** For each joint, copy `q` and perturb it by $+h$ and $-h$. Evaluate `objective()` at both configurations and store `(cost_plus - cost_minus) / (2 * h)` in `gradient[i]`.

3. **TODO 3, `inverse_kinematics()`:** Update `q` using `q - alpha * gradient`.

The supplied loop starts from MuJoCo's current joint angles and calls `objective()` to evaluate candidates using FK. It uses `h = 1e-6`, a fixed `alpha = 0.1`, and at most `5000` iterations. The loop stops when the cost is below `1e-8`.

3 Environment Setup

1. **Install Visual Studio Code:**
   Download and install Visual Studio Code. Once installed, open the Extensions tab and install the official **Python** extension provided by Microsoft.

2. **Install Python 3.11**
   This assignment requires **Python 3.11** to ensure compatibility with pre-built C++ libraries. Download the Python 3.11 installer from python.org or install it via Miniconda.
   Windows Users: During the installation process, you must check the box at the bottom of the screen that says **"Add python.exe to PATH"** before clicking Install.

3. **Install Dependencies:**
   The project relies on the following packages. Ensure they are installed in your environment before proceeding:

```
dependencies = [
    "framesviewer>=1.0.2",
    "jax>=0.5.0",
    "jaxlib>=0.5.0",
    "jaxtyping>=0.2.38",
    "matplotlib>=3.10.0",
    "mediapy>=1.2.2",
    "onnxruntime>=1.20.1",
    "playground>=0.0.3",
    "pygame>=2.6.1",
    "tensorflow>=2.18.0",
    "tf2onnx>=1.16.1",
]
```

4 Run, compare, and submit

Run the Forward Kinematics (FK) script. This script compares 100 random configurations per chain with MuJoCo. Your required maximum errors are 10^{-6} m and 10^{-5} rad:

```
python assignments/assignment_1_kinematics/forward_kinematics.py
```

Next, run the Inverse Kinematics (IK) script on the supplied trajectories. Record the printed maximum position and orientation errors over a complete run. The required maximum errors are 10^{-5} m and 10^{-4} rad.

For macOS:

```
mjpython assignments/assignment_1_kinematics/inverse_kinematics.py
```

For Windows and Linux:

```
python assignments/assignment_1_kinematics/inverse_kinematics.py
```

Submission Requirements

Submit the following items:

- Both completed Python files (`forward_kinematics.py` and `inverse_kinematics.py`).
- The three D-H tables and signs.
- The FK/IK comparison results.